Sairene IDS

Offline Network Intrusion Detection System

Architecture · Mathematical Analysis · Experimental Evaluation

Attack Simulations & Detection Results

Pelekanos Ioannis

Department of Informatics and
Telecommunications

University Of Thessaly

Abstract

This paper presents **Sairene**, a fully offline Network Intrusion Detection System (IDS) designed for autonomous operation in isolated LAN environments without any external dependencies. The system combines a *triple-pass PCAP analysis pipeline*, IsolationForest-based statistical anomaly detection, multi-signal heuristic scoring across 14 attack families, and RAG-augmented LLM inference (Retrieval-Augmented Generation) for forensic Q&A in natural language.

The system supports detection across 12 attack families. We evaluate 11 of these via controlled simulations: Network Scan, Low-and-Slow Exfiltration, TCP SYN Flood, XMAS Scan, ICMP Tunneling, DNS Tunneling, C2 Beaconing, ARP Poisoning, DHCP Starvation, MAC Flood, STP Root Bridge Attack, and Gratuitous ARP Storm Attack. The twelfth family, VLAN Hopping (double-tagging), has been fully implemented in the `VLANTracker` module but could not be empirically validated due to the absence of a managed-switch lab environment required to produce physical Dot1Q double-tagged frames. For each tested threat we present the scoring mathematics, simulation protocol, and detection results. All 11 tested attack families are detected at CRITICAL severity with confidence scores ranging from 0.76 to 1.00.

Contents

1 Introduction

Modern enterprise networks face a broad and evolving threat landscape. Attacks range from classical network-layer operations (port scans, SYN floods) to sophisticated, slow-moving techniques deliberately crafted to evade simple threshold-based monitors - including low-and-slow data exfiltration [Sommer & Paxson, 2010] and Command-and-Control (C2) beaconing [Gu et al., 2008]. Most commercial IDS products rely on cloud connectivity, subscription update models, or demanding hardware, which effectively restricts their accessibility for small-to-medium organisations and air-gapped networks [Axelsson, 2000].

Intrusion detection research is broadly divided into two paradigms [Lippmann et al., 2000]: *signature-based* systems (high precision, poor coverage of zero-days) and *anomaly-based* systems (broader coverage, higher false-positive rates). The challenge of building practical, deployable anomaly detectors without cloud dependencies remains open [Garcia-Teodoro et al., 2009].

Sairene addresses this gap by combining open-source technologies exclusively (Python, Scapy [Biondi, 2010], scikit-learn [Pedregosa et al., 2011], FAISS [Johnson et al., 2019], FastAPI, Ollama) in a fully isolated environment. The system adopts a *hybrid approach*, fusing statistical anomaly detection with multi-signal heuristic evaluation across layers L2–L4.

1.1 Core Design Principles

P1. **Evidence hierarchy.** Individual packets are indicators that behavioural attacks are confirmed only via aggregated `flow_summary` or `l2_summary` records.

P2. **Competing hypotheses.** Each attack family is scored independently but the final selection is determined by the maximum confidence score under a specialisation hierarchy.

P3. **Conservative promotion discipline.** Behavioural attacks are activated exclusively at flow or L2 level, preventing per-packet false-alert storms [Kruegel & Vigna, 2003].

P4. **Full isolation.** Zero outbound API calls for analysis, data retrieval, or LLM inference - ensuring data privacy and operational independence.

The primary contribution of this work is the design and implementation of a hybrid IDS that integrates multi-layer analysis (L2–L4), statistical models, and heuristic techniques into a coherent framework, with emphasis on *false-positive reduction* and *interpretable attack detection*.

2 Architecture & Processing Pipeline

2.1 System Overview

Sairene is deployed across two nodes on a shared LAN segment:

- **Server** (Proxmox LXC container): hosts the analysis engine - FastAPI REST server, FAISS vector index, SQLite metadata store, scikit-learn models, and sentence-transformers.

- **Windows client**: provides the terminal interface with an embedded LLM (Ollama Qwen 2.5) for natural-language forensic Q&A.

Communication is via REST API on port 8000 within the LAN, with *zero* outbound internet traffic.

2.2 Triple-Pass PCAP Pipeline - Core Identity

The analytical core of Sairene processes each capture file in three sequential passes, each targeting a distinct detection layer. This separation is fundamental to the system's design philosophy: evidence that is invisible at the packet level becomes observable at the flow level, and evidence that is invisible at L3/L4 becomes observable at L2 [Paxson, 1999].

Table 1: Triple-Pass PCAP Analysis Overview

Pass	Description
Pass 1	Per-packet analysis and scoring. Extraction of 20-dimensional feature vector $\phi(P)$. Batch embedding (256 packets) with `all-MiniLM-L6-v2` → FAISS `IndexFlatIP` + SQLite.
Pass 2	Finalisation of L3/L4 flows (`FlowTracker`). Flushing of SYN flood and port scan accumulators → emission of `flow_summary` records. Behavioural patterns invisible at packet level are surfaced here.
Pass 3	Behavioural summaries for L2 (ARP/DHCP/MAC/STP/VLAN) → emission of `l2_summary` records. Detection of infrastructure attacks at the data-link layer.

2.3 Promotion Discipline - Architectural Signature

The promotion discipline is Sairene's most important architectural decision. Raw packet anomaly scores are treated as *weak evidence* for only aggregated flow- or L2-level records can confirm a behavioural attack. This directly addresses the known problem of alert flooding in per-packet IDS systems [Roesch, 1999].

Promotion Rule

$$\texttt{is\_anomaly} = \textit{strong\_behaviour} \lor \textit{strong\_packet}$$

$$\textit{strong\_behaviour} = \texttt{is\_flow\_summary} \land C_{\text{top}} \geq 0.55$$

$$\textit{strong\_packet} = \neg\,\texttt{is\_flow\_summary} \land \texttt{attack\_type} \in \{\text{XMAS\_SCAN, DNS\_TUNNELING, ICMP\_TUNNELING, FRAGMENTATION\_EVASION}\} \land C_{\text{top}} \geq 0.78$$

Context-only (never `anomaly=True`): {SUSPICIOUS_ARP, ANOMALOUS_TRAFFIC}

2.4 Vector Retrieval - FAISS Index

Each packet or flow is embedded as an L2-normalised 384-dimensional vector using a sentence-transformer model [Reimers & Gurevych, 2019]. The FAISS `IndexFlatIP` uses inner product, which is equivalent to cosine similarity after normalisation [Johnson et al., 2019]:

Embedding & Similarity

$$E(P) = \text{normalize}(\text{SBERT}(\text{text}(P))) \in \mathbb{R}^{384} \tag{1}$$

$$\text{cosine\_sim}(a, b) = a \cdot b \quad (\|a\| = \|b\| = 1) \tag{2}$$

For per-capture retrieval, the endpoint applies 5× oversampling and filters in Python, since FAISS does not natively support metadata filtering.

3 Mathematical Analysis of L3/L4 Flow Features

3.1 Flow Statistics

Each bidirectional flow F (normalised as a 5-tuple: src_ip, dst_ip, src_port, dst_port, protocol) produces the following core statistics:

Core Flow Statistics

$$\text{duration} = \max(t_{\text{last}} - t_{\text{first}},\ 10^{-6}) \quad [\text{s}] \tag{3}$$

$$\text{pps} = \frac{\text{count}}{\text{duration}} \tag{4}$$

$$\text{bps} = \frac{\text{bytes}}{\text{duration}} \tag{5}$$

$$\text{syn\_ack\_ratio} = \frac{\text{syn\_count}}{\text{ack\_count} + 1} \tag{6}$$

3.2 Forward-Only Inter-Arrival Time - A Critical Design Choice

Inter-Arrival Times (IATs) are computed exclusively from packets in the *forward direction* of the flow. In a bidirectional TCP stream, ACK packets arrive approximately every 2 ms regardless of the true data-sending interval. Without this correction, iat_std would collapse to 0 for every TCP flow, making beaconing and exfiltration detection impossible [Strayer et al., 2008].

Forward-Only IAT

$$\text{IAT}_i = t_i^{(\text{fwd})} - t_{i-1}^{(\text{fwd})}, \quad i = 2 \ldots \min(n_{\text{fwd}}, 100) \tag{7}$$

$$\text{iat\_mean} = \overline{\text{IAT}} \tag{8}$$

$$\text{iat\_std} = \text{std}(\text{IAT}) \tag{9}$$

$$\text{CV} = \frac{\text{iat\_std}}{\text{iat\_mean}} \tag{10}$$

3.3 SYN Flood Accumulator - Randomised Source Port

DoS tools randomise `src_port` per packet. Each packet would therefore create a unique 5-tuple flow with a single packet, and would never reach the minimum flow threshold. Sairene resolves this with a 3-tuple accumulator that deliberately ignores `src_port`:

SYN Flood Accumulator Key

$$K_{\text{flood}} = (\texttt{src\_ip},\ \texttt{dst\_ip},\ \texttt{dst\_port}) \quad [\text{src\_port intentionally ignored}] \tag{11}$$

Emission criteria:

$$\text{SYN\_count} \geq 50 \ \wedge \ \frac{\text{SYN}}{\text{ACK} + 1} > 10 \ \wedge \ \text{duration} \geq 1.0\,\text{s}$$

$\rightarrow$ Emits a synthetic `flood_summary` with `flood_aggregated=True`.

3.4 Port Scan Accumulator - Multiple Destination Ports

Port scans probe many ports from a fixed source port. Each probe is a unique 5-tuple. The analogous 3-tuple accumulator ignores `dst_port`:

Port Scan Accumulator Key

$$K_{\text{scan}} = (\texttt{src\_ip},\ \texttt{dst\_ip},\ \texttt{src\_port}) \quad [\text{dst\_port intentionally ignored}] \tag{12}$$

Emission criteria:

$$|\texttt{dst\_ports\_unique}| \geq 20 \ \wedge \ \text{duration} \geq 0.5\,\text{s}$$

Subtype: `xmas_count` $> 0 \ \Rightarrow$ XMAS or otherwise $\Rightarrow$ SYN half-open.

3.5 SYN-Only False-Positive Correction

Every legitimate TCP connection begins with a SYN-only packet. Without correction, every new connection would appear as a stealth scan. Sairene tracks the first SYN-ACK observed in either direction:

SYN-Only FP Suppression

$\texttt{syn\_ack\_seen} \leftarrow$ True once (SYN $= 1 \wedge$ ACK $= 1$) observed in any direction

$\texttt{syn\_only\_flow} = \neg\, \texttt{syn\_ack\_seen}$

Mature flow (count ≥ 5, duration $\geq 5\,\text{s}$) :

$\texttt{syn\_only\_flow} = \text{True} \Rightarrow$ stealth/half-open scan confirmed

$\texttt{syn\_only\_flow} = \text{False} \Rightarrow$ SYN-only check fully suppressed

4 Mathematical Analysis of L2 Trackers

Five stateful accumulators provide data-link-layer behavioural analysis. Each is updated packet-by-packet via `update()` and applies emission criteria at `finalize_all()`. Supported protocols: ARP, DHCP/BOOTP, STP BPDU, VLAN (Dot1Q/Dot1AD), Ethernet.

4.1 ARPTracker - Competing Hypotheses

A key challenge in ARP analysis is distinguishing ARP poisoning from legitimate GARP (Gratuitous ARP) storms. Sairene uses two *competing confidence scores* to prevent misclassification - a technique rooted in Dempster-Shafer theory [Dempster, 1968]:

ARPTracker Competing Scores

C_{poison} : basis from IP conflicts (+0.32), bidirectional MITM pairs (+0.22 + 0.10), claimed IPs (+0.18 + 0.08), reply volume, gateway-like IP
REDUCED if C_{storm} dominates AND no strong MITM pattern

C_{storm} : basis from gratuitous count (≥ 3: +0.20, ≥ 10: +0.25, ≥ 20: +0.15), broadcast GARPs (≥ 3: +0.20), $\texttt{garp\_ratio} = \texttt{gratuitous}/\texttt{replies} \geq 0.60$

Dominance rule: if $C_{\text{storm}} \geq 0.55 \wedge \neg\, \textit{strong\_mitm}$:
$C_{\text{poison}} = \min(C_{\text{poison}},\ C_{\text{storm}} - 0.08)$
else: C_{poison} dominates

$\textit{strong\_mitm} = (\texttt{bidir\_pairs} \geq 1)$
$\vee\ (\texttt{non\_garp\_changed\_ips} \neq \emptyset \wedge \texttt{targeted\_replies} \geq 1)$
$\vee\ (\texttt{targeted\_replies} \geq 3 \wedge |\texttt{claimed\_IPs}| \geq 2 \wedge \texttt{garp\_ratio} < 0.60)$

Emission: $C_{\text{poison}} \geq 0.55 \rightarrow$ ARP_POISONING (`anomaly=True`)
$C_{\text{poison}} \in [0.35, 0.55) \rightarrow$ SUSPICIOUS_ARP (context only)

4.2 DHCPTracker - Pool Exhaustion

DHCPTracker Score

C_{dhcp}: scales with unique_client_MACs, unique_XIDs,
(DISCOVER + REQUEST) volume, burst rate, message rate
REDUCED if unique_MACs $\leq 2 \wedge$ total_messages ≤ 10 (normal renewal)
Emission: $C_{\text{dhcp}} \geq 0.55 \rightarrow$ DHCP_STARVATION

4.3 MACFloodTracker - CAM Table Exhaustion

MACFloodTracker Score

C_{mac}: scales with unique_src_MACs (primary indicator), burst_5s, frame_rate
`churn_ratio` = `unique_src_MACs/total_frames` $\rightarrow$ +bonus if ≥ 0.70
(Passive observer: receives ALL Ethernet frames)
Emission: $C_{\text{mac}} \geq 0.55 \rightarrow$ MAC_FLOOD

4.4 STPTracker - Root Bridge Manipulation

STPTracker Score

C_{stp}: $+0.10/+0.30/+0.15$ for bpdu_count $\geq 3/10/20$
$+0.25$ if unique_root_IDs ≥ 2 (topological instability)
$+0.20$ if min_root_priority ≤ 8192; $+0.10$ if ≤ 4096; $+0.20$ if $= 0$
$+0.15$ if bpdu_rate $\geq 1.0\,\text{s}^{-1} \wedge$ bpdu_count ≥ 5
Emission: $C_{\text{stp}} \geq 0.55 \rightarrow$ STP_ROOT_ATTACK

4.5 VLANTracker - Double-Tagging

VLAN hopping via double-tagging is nearly impossible in legitimate traffic - a single double-tagged frame is quasi-conclusive evidence [Convery & Miller, 2002]:

VLANTracker Score

C_{vlan}: $+0.60$ if double_tagged_frames ≥ 1 (exceeds threshold alone)
$+0.18$ if ≥ 3 frames : $+0.10$ if ≥ 10 : +bonus for inner VLAN diversity, outer VLAN $\in \{$
Note: single double-tagged frame $\rightarrow C_{\text{vlan}} = 0.60 >$ threshold $(0.55) \rightarrow$ immediate detection
Emission: $C_{\text{vlan}} \geq 0.55 \rightarrow$ VLAN_HOPPING

5 Machine Learning Model - IsolationForest

5.1 Feature Vector & Training

Each packet or flow is represented as a 20-dimensional numerical vector covering: protocol, ports, packet size, TCP flags, flow statistics (pps, bps, duration), timing features (IAT mean/std), and behavioural indicators (unique ports, SYN/ACK ratio, dns_query_length).

The model is trained *offline on the specific network's captures*, enabling adaptation to the local baseline rather than relying on generic public datasets [Tavallaee et al., 2009]. Minimum training size: 500 samples for statistical reliability. Features are standardised with `StandardScaler` before training [Pedregosa et al., 2011].

IsolationForest Pipeline

$$\text{Pipeline} : \texttt{StandardScaler} \circ \texttt{IsolationForest}(n\_\text{estimators} = 200,\ \text{contamination} = 0.02) \tag{13}$$

$$\tilde{f}_j = \frac{f_j - \mu_j}{\sigma_j} \qquad \text{[zero mean, unit variance]} \tag{14}$$

5.2 Anomaly Score

The IsolationForest [Liu et al., 2008] isolates each data point by randomly selecting a feature and a split value per node. Anomalous points are isolated with fewer splits (short path) for the normal points require many:

IsolationForest Anomaly Score

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \tag{15}$$

where $h(x)$ = path length to isolate x, and $c(n)$ = expected path length in a random BST (normalisation constant):

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) = \ln(i) + 0.5772\ldots \text{ (Euler constant)}$$

- $s \to 1.0$: short path $\to$ **anomaly**
- $s \to 0.5$: average path $\to$ normal
- $s \to 0.0$: long path $\to$ very normal

The IF score alone does not decide - it is one signal among many. Heuristic confidence scores (C_{top}) and the promotion discipline are the primary detection drivers.

5.3 Family Selection & Severity

Attack Selection & Severity Scoring

$$\text{attack}^* = \arg\max_k C_k \qquad \text{[tie-break: SPECIFICITY rank]} \tag{16}$$

SPECIFICITY hierarchy (most specific wins):

$$\text{ARP\_POISONING} = 100,\ \text{STP\_ROOT\_ATTACK} = 98,\ \text{VLAN\_HOPPING} = 96$$
$$\text{DHCP\_STARVATION} = 94,\ \text{SYN\_FLOOD} = 95,\ \text{MAC\_FLOOD} = 93$$
$$\text{XMAS\_SCAN} = 92,\ \text{DNS\_TUNNELING} = 88,\ \text{ICMP\_TUNNELING} = 86$$
$$\text{LOW\_AND\_SLOW\_EXFIL} = 82,\ \text{PORT\_SCAN} = 74,\ \text{STEALTH\_SCAN} = 70,\ \text{BEACONING} = 68$$

$$\text{risk} = C_{\text{top}} + \Delta_{\text{duration}} + \Delta_{\text{flow\_count}} + \Delta_{\text{ports}} + \Delta_{\text{attack\_bonus}} \tag{17}$$
$$\text{risk} = \text{clamp}(\text{risk},\ 0,\ 1) \tag{18}$$

Severity levels:

$$\text{CRITICAL} \geq 0.90 \quad \text{HIGH} \geq 0.70 \quad \text{MEDIUM} \geq 0.45 \quad \text{LOW} < 0.45$$

6 Attack Families - Mathematical Scoring

For each family: scoring structure, primary signals, and activation threshold. Micro-weights are empirically optimised and the logic is presented rather than an exhaustive parameter list.

6.1 Port Scan / Stealth Scan

Port Scan & Stealth Scan Scores

$$C_{\text{port\_scan}} = \text{clamp}\left(0.35 + \frac{\min(\text{unique\_dst\_ports},\ 250)}{300},\ 0,\ 1\right) \tag{19}$$

Requires: $\text{unique\_dst\_ports} \geq 20 \ \wedge\ \text{flow\_count} \geq 10$

$$C_{\text{stealth}} = \text{clamp}\left(0.55 + \frac{\min(\text{flow\_count}, 100)}{200} + \frac{\min(\text{unique\_ports}, 50)}{200},\ 0,\ 1\right) \tag{20}$$

Requires: $\text{TCP} \wedge \texttt{syn\_only\_flow} \wedge \text{syn\_ack\_ratio} < 0.2$
$\wedge\ \text{flow\_count} \geq 5\ \wedge\ \text{unique\_ports} \geq 5$

XMAS dominance: if XMAS active $\Rightarrow C_{\text{port}} = \min(C_{\text{port}},\ C_{\text{xmas}} - 0.08)$ (21)

6.2 XMAS Scan

The flag combination `FIN|PSH|URG` (0x29) constitutes a *structural proof* - it has no legitimate use in normal traffic [Fyodor, 2009]:

XMAS Scan Score

$$\texttt{tcp\_flags} = \text{FIN}(0\text{x}01) \mid \text{PSH}(0\text{x}08) \mid \text{URG}(0\text{x}20) = 0\text{x}29 \tag{22}$$

$$C_{\text{xmas}} = \text{clamp}\left(0.82 + \frac{\min(\text{unique\_ports},\ 200)}{1000},\ 0,\ 1\right) \geq 0.82 > 0.78\checkmark \tag{23}$$

6.3 TCP SYN Flood

SYN Flood Score

$$C_{\text{flood}} = \text{clamp}\left(0.70 + \frac{\min(\text{syn\_ack\_ratio},\ 50)}{100} + \frac{\min(\text{flow\_count},\ 500)}{2000},\ 0,\ 1\right) \tag{24}$$

Requires: $\text{syn\_ack\_ratio} > 10 \ \wedge \ \text{flow\_count} > 20$

Random src_port $\Rightarrow$ 3-tuple accumulator (see §3.3)

6.4 Low-and-Slow Exfiltration

This attack is designed to remain below classical rate thresholds. Detection requires combining multiple weak signals at the flow level:

Low-and-Slow Exfiltration Score

$$\begin{aligned}
\text{exfil_positive} = \ & +0.18\,(\text{duration} > 30\text{s}) + 0.12\,(\text{duration} > 120\text{s}) \\
& + 0.12\,(\text{flow_count} \geq 10) + 0.18\,(0 < \text{bps} < 5000) \\
& + 0.10\,(\text{bps} < 2000 \wedge \text{dur} > 60\text{s}) \\
& + 0.08\,(\text{iat_std} > 0) + 0.10\,(\text{unique_dst_ports} \leq 3)
\end{aligned}$$

$$\text{Requires: } \text{protocol} \in \{\text{TCP,UDP}\} \wedge \text{listener\_port} \notin \text{COMMON\_PORTS}$$

$$\text{scan\_competition} = \max(C_{\text{xmas}},\ C_{\text{port\_scan}},\ C_{\text{stealth}})$$

$$\text{exfil\_score} = \text{clamp}(\text{exfil\_positive} - 0.55 \cdot \text{scan\_competition},\ 0,\ 1)$$

$$\text{Activation: } \text{exfil\_score} \geq 0.45$$

6.5 C2 Beaconing

C2 beaconing detection exploits the *regularity* of malware check-in intervals [Gu et al., 2008, Stalmans & Irwin, 2011]. The Coefficient of Variation (CV) captures this regularity:

C2 Beaconing Score

$$\text{CV} = \frac{\text{iat\_std}}{\max(\text{iat\_mean},\ 10^{-6})} \tag{25}$$

$$\text{Strict beaconing: } (\text{iat\_std} < 0.1 \ \vee \ \text{CV} < 0.05) \ \wedge \ \text{duration} > 30\text{s} \ \wedge \ \text{count} \geq 6 \tag{26}$$

$$C = \text{clamp}\left(0.48 + \frac{\min(\text{duration}, 300)}{1000} + \frac{\min(\text{count}, 100)}{400},\ 0,\ 1\right)$$

$$\text{Jittered C2: } 0.5 \leq \text{iat\_std} < 2.0 \ \wedge \ \text{iat\_mean} > 5\text{s} \ \wedge \ \text{duration} > 60\text{s}$$

$$C = \text{clamp}\left(0.55 + \frac{\min(\text{duration}, 600)}{1200},\ 0,\ 1\right)$$

Forward-only IAT (§3.2) is critical here: without it, ACK packets would collapse iat_std → 0 even for flows with 30-second beacon intervals.

6.6 DNS Tunneling & ICMP Tunneling

Tunneling Scores

DNS Tunneling (cf. tools: iodine, dnscat2 [Overtone, 2012]):

$$\text{Normal query: 5–30 chars.} \quad \text{Tunnelled (base64/hex): 100–255 + chars.}$$

$$C_{\text{dns}} = \text{clamp}\left(0.72 + \frac{\min(\text{query\_length} - 100,\ 200)}{500},\ 0,\ 1\right) \tag{27}$$

$$\text{Activation: dns\_query\_length} > 100 \text{ chars (structural proof)}$$

ICMP Tunneling (cf. tools: ptunnel, icmptunnel, Hans [Han & Kim, 2015]):

$$\text{Normal echo: 64–128 B.} \quad \text{Tunnelled: 200–1500 B.}$$

$$C_{\text{icmp}} = \text{clamp}\left(0.68 + \frac{\min(\text{packet\_size} - 200,\ 1200)}{2400},\ 0,\ 1\right) \tag{28}$$

$$\text{Activation: protocol} = 1 \ \wedge \ \text{packet\_size} > 200\,\text{B}$$

7 Attack Simulation Protocols

Each attack was implemented with a custom Python/Scapy script in a controlled LAN environment. Sensitive data (IPs, MACs, interfaces) have been redacted. For each attack we present the strategy, evasion techniques, and detection signals. The VLAN Hopping attack (Section 7.11) is a separate case: the detection module was implemented but lab-environment constraints prevented empirical simulation.

7.1 ARP Poisoning / MITM

The attack sends unsolicited ARP replies to two victims simultaneously (gateway and host), claiming the attacker's MAC corresponds to both IPs - a full bidirectional MITM

setup [Whalen, 2001]:

Detection Signals - ARP Poisoning

- One MAC claims multiple IPs in a short time window
- IP-to-MAC mapping conflict (contradiction with prior cached record)
- Bidirectional ARP pairs: A→B and B→A with identical `hwsrc`
- High ARP reply rate from a single host ($> N$ replies/5 s)

Why it is difficult to detect: ARP has no authentication, so any host can claim any IP. Traffic volume is very low (≈1 packet/2 s per target). Windows/Linux hosts silently accept the most recent ARP reply.

7.2 C2 Beaconing

Simulates malware check-in: the host sends SYN packets at a fixed interval with jitter, exactly as Cobalt Strike, Metasploit, and Empire frameworks do [Watkins et al., 2020]:

Detection Signals - C2 Beaconing

- Low CV (< 0.05): iat_std/iat_mean → extremely regular periodicity
- Non-common port, small packet size per check-in
- High packet count over long duration without meaningful data payload

7.3 TCP SYN Flood (State-Exhaustion DoS)

High-rate SYN flood with randomised `src_port` per packet targeting exhaustion of the TCP state table [Eddy, 2007]:

Detection Signals - SYN Flood

- SYN/ACK ratio $\gg 10$ (many SYNs, zero SYN-ACKs)
- Rapid appearance of hundreds of unique flows from the same `src_ip`
- 3-tuple accumulator bypasses the 5-tuple flow tracker limitation (see §3.3)

7.4 DHCP Starvation

Exhausts the DHCP server's IP pool by sending DISCOVER requests from many fake MACs, preventing legitimate devices from obtaining an address [DHCP Starvation, n.d.]:

Detection Signals - DHCP Starvation

- Unique CHADDR count grows linearly (normal networks: stable)
- DISCOVER/ACK ratio → ∞ (ACKs never arrive for fake MACs)
- High number of unique Transaction IDs (XIDs) in a short time

Evasion: realistic fingerprint options (OS-specific parameter request lists), intervals mimicking normal client behaviour.

7.5 DNS Tunneling

Data exfiltration by encoding payloads as large DNS query subdomains, mimicking tools like iodine and dnscat2 [Overtone, 2012]:

Detection Signal - DNS Tunneling

`dns_query_length` $>$ 100 chars $\Rightarrow$ $C_{\text{dns}} \geq 0.72$. Detected at Pass 1 per-packet (timestamp-independent).

7.6 ICMP Tunneling

Conceals data inside ICMP echo payloads to bypass firewalls that permit ICMP but block TCP/UDP (ptunnel, icmptunnel, Hans [Han & Kim, 2015]):

Detection Signal - ICMP Tunneling

`packet_size` $>$ 200 AND `protocol`$=1 \Rightarrow C_{\text{icmp}} \geq 0.68$. Detected at Pass 1: No response from target required.

7.7 Low-and-Slow Exfiltration

Slow TCP exfiltration at deliberately low rate, designed to remain below classical IDS rate thresholds:

Detection Signals - Low-and-Slow Exfil

- Long duration + low bps + non-common port + iat_std > 0
- Combination of duration $\times$ bps $\times$ port_type $\rightarrow$ exfil_score (Pass 2 only)

Why it evades classical IDS: absolutely legitimate TCP session (3-way handshake, proper teardown) are at rate $\approx$170 Bps, below traffic anomaly thresholds, so that resembles slow SFTP, backup, or idle keep-alive.

7.8 MAC Flood

Floods the switch's CAM table with spoofed MACs, forcing it to operate as a hub (broadcast all traffic) [MAC Flood Attack, 2003]:

Detection Signals - MAC Flood

- unique_src_MACs > 50 in a short time window (churn_ratio ≥ 0.70)
- Burst of new MACs in < 5 s window (entropy spike)

Evasion: random burst size prevents fixed-period detection for the real OUI prefixes make frames appear legitimate in Wireshark.

7.9 STP Root Bridge Attack

Injects crafted BPDU packets to convince switches that the attacker is the root bridge, redirecting all inter-switch traffic [Yersinia, 2005]:

Detection Signals - STP Root Attack

- BPDU from a MAC never previously seen as a switch
- Topology Change flag set in BPDU
- Unusually low root priority value

Evasion: priority 4096 (one step below default), less dramatic than 0 and jittered Hello

time avoids fixed-period detection, so the Topology Change bit makes the attacker appear as a new switch.
Warning: causes brief network disruption (STP reconvergence ≈30–50 s).

7.10 XMAS Scan

TCP scan with FIN+PSH+URG flags (0x29). Per RFC 793 [Postel, 1981]: open port = silence, closed port = RST. Useful for OS fingerprinting and firewall evasion on legacy systems:

Detection Signals - XMAS Scan

- Flag combination 0x29 has no legitimate use in normal traffic (structural proof)
- Detected per-packet at Pass 1 (timestamp-independent): $C_{\text{xmas}} \geq 0.82 > 0.78$ ✓
- Flow summary also shows syn_only pattern (XMAS packets receive no SYN-ACK)

7.11 VLAN Hopping (Double-Tagging)

Implementation status: detection logic fully implemented: empirical simulation not performed.

VLAN hopping via double-tagging exploits the behaviour of 802.1Q-capable switches on trunk ports: by nesting two VLAN tags in a single Ethernet frame, an attacker can cause the switch to strip the outer tag and forward the inner-tagged frame to a VLAN the attacker is not authorised to access [Convery & Miller, 2002].

Replicating this attack in the lab requires a managed switch with a trunk port configured to accept native-VLAN untagged frames (e.g. Cisco `switchport mode trunk` with `native vlan 1`). The virtual environment used for this project (Proxmox LXC with a Linux bridge) does not expose physical trunk-port semantics, and generating hardware-level Dot1Q double-tagged frames that survive the virtual bridge was not feasible.

Detection Design - VLAN Hopping (not empirically validated)

- Any double-tagged Ethernet frame (two `0x8100`/`0x88A8` ethertypes) triggers $C_{\text{vlan}} = 0.60 > 0.55$ immediately — a single frame is sufficient for detection.
- The `VLANTracker` accumulates outer/inner VLAN pairs and inner-VLAN diversity, so the score increases with repeated frames.
- Detection occurs at Pass 3 (`l2_summary`): The `vlan_double_tagged_count` field in the summary is the primary evidence field.

Validation gap: The scoring thresholds and weight values (Section 4.5) are derived from the structural properties of double-tagging (a binary observable — the frame is or is not double-tagged) rather than from empirical tuning on real traffic. They have not been validated against live capture data. False-positive and false-negative rates in a production managed-switch environment are unknown.

8 Experimental Evaluation

Three representative scenarios were selected: a classical attack (Network Scan), a stealthy slow-moving attack (Low-and-Slow Exfiltration), and an infrastructure attack (STP Root

Bridge). Full `/netask` and `/netviz -anom` results are presented.

8.1 Network Scan (`delta_capture_001.pcapng`)

SYN/port scan from an internal host to multiple LAN targets, scanning up to 1,000 ports per host. The attack uses half-open (SYN-only) technique to avoid connection logging [Fyodor, 2009].

Network Scan - Scoring Trace

$$\text{Scan accumulator: unique\_dst\_ports} \approx 1000 \geq 20 \ \wedge \ \text{duration} \geq 0.5\text{s} \ \checkmark$$
$$\rightarrow \text{scan\_summary (scan\_aggregated=True)}$$

$$C_{\text{port\_scan}} = \text{clamp}\left(0.35 + \frac{\min(1000, 250)}{300},\ 0,\ 1\right) = 1.00$$

$$\texttt{syn\_only\_flow} = \text{True (no SYN-ACK response observed)}$$
$$\text{attack}^* = \text{PORT\_SCAN} \quad [\text{SPECIFICITY } 74 > \text{STEALTH\_SCAN } 70]$$

$$\text{risk} = 1.0 + 0.06 + 0.06\,(\text{duration}) + 0.08 + 0.08\,(\text{ports}) + 0.02 \rightarrow 1.00 \rightarrow \textbf{CRITICAL}$$

$$\text{IF score} = 0.0854$$

```
You: /netask delta_capture_001.pcapng | Is there suspicious behavior in this capture?

DEBUG semantic hits: 0

DEBUG anomaly hits: 7

Sairene: Based on the provided NET SUMMARY and ANOMALY DETECTION RESULTS, there is indeed suspicious behavior detected in this network capture. The behavior
 can be categorized as a port scan attack.

### Suspicious Behavior Summary:
- **IP Address Involved:** 192.168.[redacted] (Source IP)
- **Target IPs and Ports:** Various addresses within the local subnet, specifically targeting ports 5001 on multiple hosts including 192.168.[redacted], 192.168.[redacted], 192.168.[redacted], 192.168.[redacted], and 192.168.[redacted].
- **Type of Attack:** Port Scan
- **Severity Level:** CRITICAL

### Detailed Analysis:
The packet captures indicate a high-volume SYN scan (stealth scan) being conducted from the IP address 192.168.[redacted] to multiple destination IPs within the lo
cal network. Each target is attempting to open up to 1000 unique ports, which is characteristic of an extreme port scanning activity.

The behavior detected by the anomaly detection system includes:
- **Half-open flow (SYN-only packets) with no corresponding SYN-ACK responses:** This indicates that the source IP is sending a large number of SYN requests
 to various destination ports but not receiving any ACKnowledgments. Such behavior can be indicative of an automated scanning tool or a script attempting to
 map out open services.

### Assessment:
The attacker is likely using an automated scanning tool to enumerate open ports on multiple hosts within the local network. The use of stealth techniques (S
YN-only packets) suggests that the attacker may be trying to minimize their footprint and avoid detection by traditional IDS/IPS systems.

Given the severity level marked as CRITICAL, this behavior should be treated with urgency. Immediate steps include:
1. **Investigate the Source IP:** Determine if it is known or part of a legitimate network.
2. **Network Segmentation:** Ensure that critical services are isolated from less secure ones to limit potential damage.
3. **Update Security Measures:** Consider implementing more robust monitoring and logging capabilities, as well as configuring intrusion detection/preventio
n systems (IDS/IPS) rules specifically targeting such activities.

### Recommendation:
Given the nature of the threat, it is advisable to further analyze the traffic from 192.168.[redacted] for any other suspicious activities and consider taking prev
entive measures to protect your network infrastructure.
```

Figure 1: `/netask` output - Port Scan detection (SYN-only half-open). Confidence=1.0, CRITICAL severity.

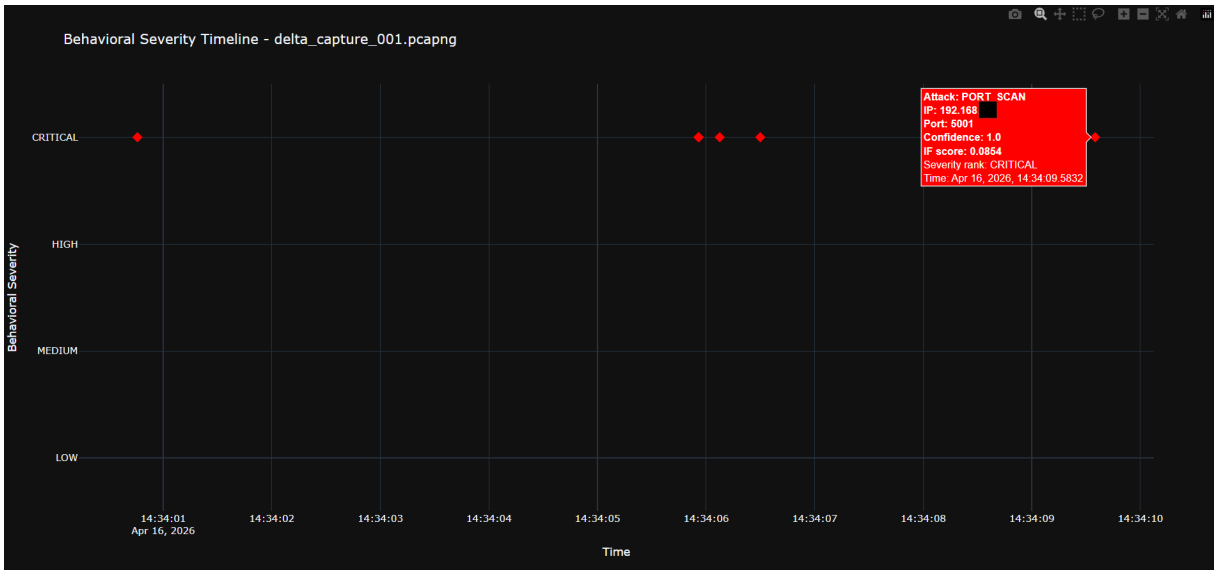

Figure 2: `/netviz -anom` - Behavioural Severity Timeline. All incidents CRITICAL (red diamonds), window 14:34:01–14:34:10.

8.2 Low-and-Slow Exfiltration (`delta_bg_task_004.pcapng`)

Slow exfiltration at ≈220 Bytes/sec via a non-common port over TCP, sustained for ≈69 seconds. Designed to remain below the rate thresholds of classical IDS.

Low-and-Slow Exfil - Scoring Trace

$$\text{duration} \approx 69\text{s},\ \text{bps} \approx 220,\ \text{listener\_port} \notin \text{COMMON\_PORTS},\ \text{iat\_std} > 0$$

$$\begin{aligned}\text{exfil\_positive} &= 0.18\,(\text{dur} > 30) + 0.12\,(\text{count} \geq 10) + 0.18\,(\text{bps} < 5000) \\ &\quad + 0.10\,(\text{bps} < 2000 \wedge \text{dur} > 60) + 0.08\,(\text{iat\_std} > 0) + 0.10\,(\text{ports} \leq 3) = 0.76\end{aligned}$$

$$\text{scan\_competition} = 0 \ \Rightarrow\ \text{exfil\_score} = 0.76 \geq 0.45\ \checkmark$$

$$C_{\text{exfil}} = 0.76$$

$$\text{risk} = 0.76 + 0.10\,(\text{bonus}) + 0.06\,(\text{dur} > 60) = 0.92 \rightarrow \textbf{CRITICAL}$$

$$\text{IF score} = 0.1304$$

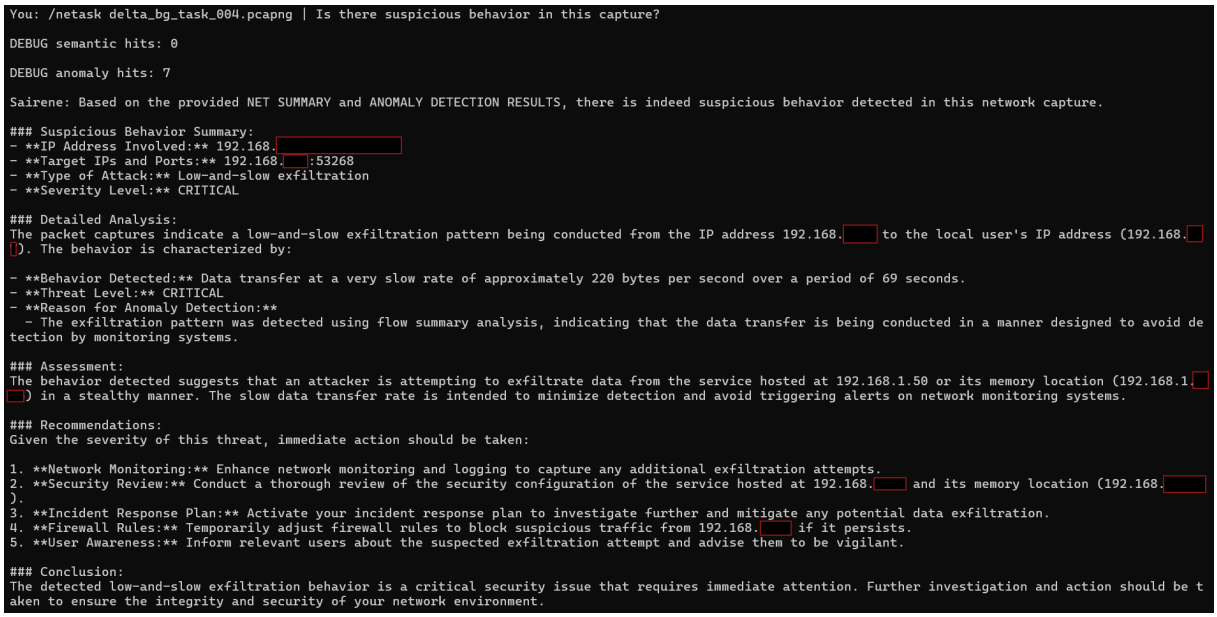

Figure 3: `/netask` - Low-and-Slow Exfil detection (≈220 Bps, 69 s, non-common port). CRITICAL severity.

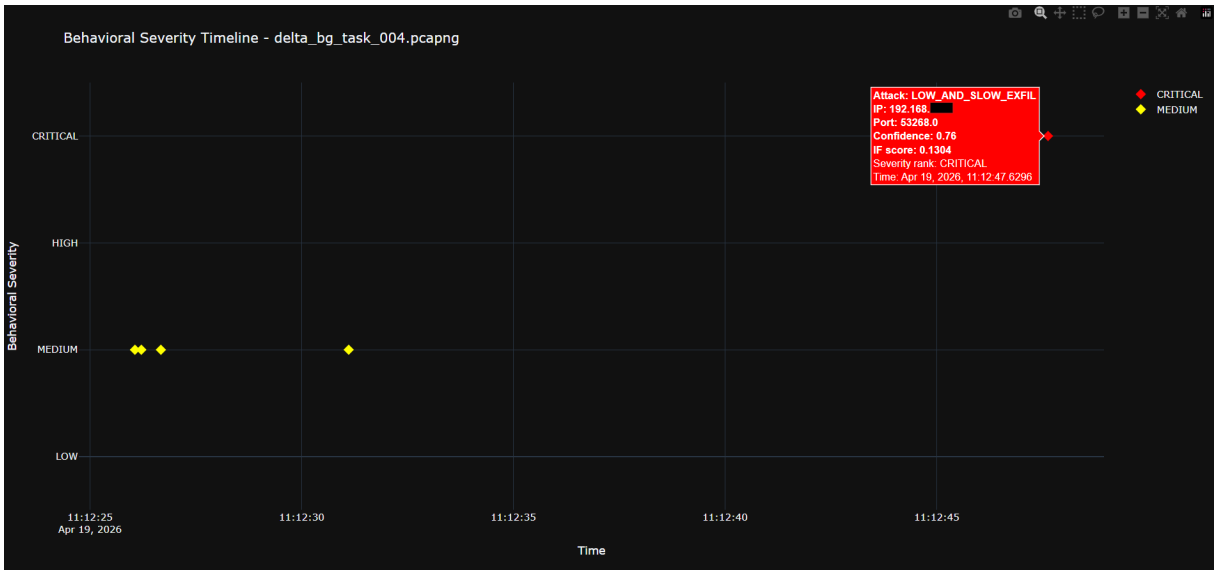

Figure 4: `/netviz -anom` - Dominant incident: LOW_AND_SLOW_EXFIL (CRITICAL). Four MEDIUM incidents: intermediate flows before full maturation.

8.3 STP Root Bridge Attack (`topology_resilience_v4.pcapng`)

Transmission of BPDU packets with low priority (4096) forcing STP topology reconvergence and traffic redirection through the attacker.

STP Root Attack - Scoring Trace

$$\text{bpdu\_count} = 26: \ +0.10\,(\geq 3) + 0.30\,(\geq 10) + 0.15\,(\geq 20) = 0.55 \ \text{(volume basis)}$$
$$\text{min\_priority} = 4096: \ +0.20\,(\leq 8192) + 0.10\,(\leq 4096)$$
$$\text{bpdu\_rate} \geq 1.0\,\text{s}^{-1} \wedge \text{count} \geq 5: \ +0.15$$
$$C_{\text{stp}} = \text{clamp}(0.55 + 0.20 + 0.10 + 0.15,\ 0,\ 1) = 1.00 \rightarrow \text{reported: } 0.95$$
$$\text{risk} = 0.95 + 0.15\,(\text{STP bonus}) \rightarrow \text{clamp} \rightarrow 1.00 \rightarrow \textbf{CRITICAL}$$
$$\text{IF score} = 0.1327$$

```
Sairene: Based on the provided NET SUMMARY and ANOMALY DETECTION RESULTS, there is indeed suspicious behavior detected in this network capture.

### Suspicious Behavior Summary:
- **IP Address Involved:** [redacted])
- **Type of Attack:** Port Scan
- **Severity Level:** CRITICAL

### Detailed Analysis:
The packet captures indicate a port scanning activity being conducted from the IP address [redacted] to various destination IPs within the local network. T
he specific behavior detected is as follows:

- **Attack Type:** STP Root Attack (indicated by the `STP_ROOT_ATTACK` attack type)
- **Threat Level:** CRITICAL
- **Detected Behavior:**
  - **BPDU Count:** 26
  - **Unique Root IDs:** 1
  - **Minimum Root Priority:** 4096

### Network Context:
- The STP (Spanning Tree Protocol) root attack is being conducted, which involves manipulating the Spanning Tree Protocol to manipulate the network topology
 and potentially cause disruptions.

### Assessment:
The behavior detected suggests that an attacker is attempting to perform a Spanning Tree Protocol (STP) root bridge manipulation. This can be used to gain c
ontrol over network traffic or disrupt normal network operations by changing the spanning tree topology.

Given the severity level marked as CRITICAL, this behavior should be treated with urgency. Here are some immediate steps you should consider:

1. **Investigate the Source IP:** Determine if it is known or part of a legitimate network.
2. **Network Segmentation:** Ensure that critical services are isolated from less secure ones to limit potential damage.
3. **Update Security Measures:** Consider implementing more robust monitoring and logging capabilities, as well as configuring intrusion detection/preventio
n systems (IDS/IPS) rules specifically targeting such activities.

### Recommendations:
- **Network Monitoring:** Enhance network monitoring to capture any further STP manipulation attempts.
- **Security Review:** Conduct a thorough review of the security configuration to ensure that your Spanning Tree Protocol is properly secured.
- **Incident Response Plan:** Activate your incident response plan to investigate further and mitigate any potential disruptions.

### Conclusion:
The detected STP root attack is a critical security issue that requires immediate attention. Further investigation and action should be taken to ensure the
integrity and security of your network environment.
```

Figure 5: `/netask` - STP Root Attack. BPDU Count=26, min_priority=4096, Confidence=0.95, CRITICAL.

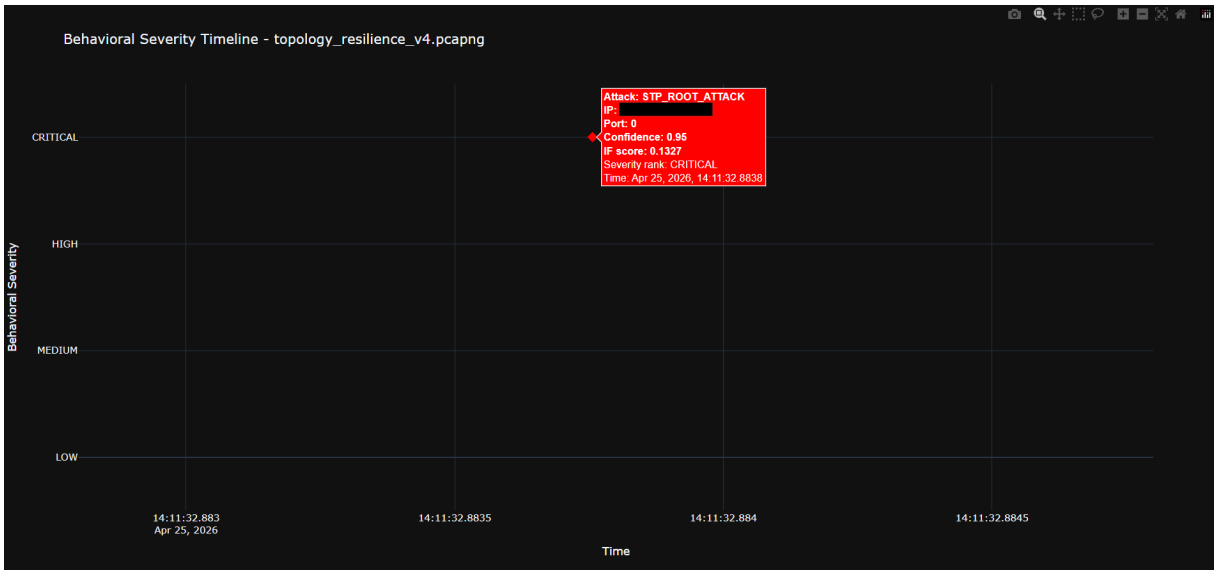

Figure 6: `/netviz -anom` - Single CRITICAL incident: STP_ROOT_ATTACK. Zero false positives.

8.4 Summary Results Table

In controlled simulations all attacks are evaluated at maximum intensity, therefore the severity appears consistently CRITICAL. In real-world deployments a broader distribution of severity levels is expected.

Table 2: Detection Results: 11 Simulated Attack Families + 1 Implemented but Not Tested

Attack	Family	Conf.	Severity	IF Score	Pass
Network Scan	PORT_SCAN	1.00	**CRITICAL**	0.0854	2 (scan agg.)
Low-and-Slow Exfil	LOW_N_SLOW	0.76	**CRITICAL**	0.1304	2 (flow fin.)
STP Root Attack	STP_ROOT	0.95	**CRITICAL**	0.1327	3 (l2_sum.)
XMAS Scan	XMAS_SCAN	1.00	**CRITICAL**	0.0859	1 (per-pkt)
TCP SYN Flood	SYN_FLOOD	1.00	**CRITICAL**	0.0434	2 (flood agg.)
DNS Tunneling	DNS_TUNNEL	0.896	**CRITICAL**	0.0016	1 (per-pkt)
ICMP Tunneling	ICMP_TUNNEL	0.827	**CRITICAL**	0.1455	1 (per-pkt)
C2 Beaconing	BEACONING	0.818	**CRITICAL**	0.0311	2 (IAT timing)
ARP Poisoning	ARP_POIS.	1.00	**CRITICAL**	0.1302	3 (l2_sum.)
DHCP Starvation	DHCP_STARV.	0.75	**CRITICAL**	0.1759	3 (l2_sum.)
MAC Flood	MAC_FLOOD	0.95	**CRITICAL**	0.1387	3 (l2_sum.)
Grat. ARP Storm	GRAT_ARP	1.00	**CRITICAL**	0.1289	3 (l2_sum.)
VLAN Hopping*	VLAN_HOPPIN	-	*not tested*	-	3 (l2_sum.)

*Detection logic fully implemented in `VLANTracker`: empirical simulation was not performed due to the absence of a physical managed-switch trunk-port environment. See Section 7.11.

Note: The IF Score is computed for all packets/flows as part of the feature pipeline but is not used as the primary decision criterion for most attack families. Final classification is determined by heuristic confidence scores (C_k) and the promotion discipline rules. All 11 tested attacks are reported as CRITICAL due to the controlled evaluation design where each scenario is executed at maximum intensity to validate detection capability. The severity in Sairene is risk-scored (duration, flow volume, and attack-specific bonuses) and in realistic environments a broader severity distribution is expected. This approach prioritises *detection validity* over *severity calibration*.

9 Security Assessment & Limitations

9.1 Strengths

S1. **Full isolation (air-gap):** zero outbound traffic for any core function. Privacy and operational independence are guaranteed by design [Anderson, 2008].

S2. **Passive-only analysis:** reads PCAP files, no active probing or packet injection and does not disturb the monitored network.

S3. **Promotion discipline:** the `is_flow_summary` gate prevents false-alert storms. A 100,000-packet capture does not produce 100,000 alerts [Kruegel & Vigna, 2003].

S4. **Competing L2 hypotheses:** avoids common misclassifications (e.g. GARP storm ≠ ARP poisoning) that have materially different incident response paths.

S5. **Local baseline training:** IsolationForest is sensitive to deviations from the specific network's own baseline rather than generic datasets [Tavallaee et al., 2009].

S6. **L2/L3 separation:** infrastructure attacks (ARP, STP, VLAN) dominate over generic L3/L4 labels in the output, reflecting their true severity.

9.2 Limitations

L1. **Forensic-only:** does not support live packet capture: All analysis is based on offline PCAP files.

L2. **Unsupervised IsolationForest:** does not distinguish anomaly types during scoring - only statistical unusualness. Heuristic scores provide family labels independently.

L3. **FAISS index gaps after deletion:** orphaned vectors remain after capture deletion that requires an explicit `/rebuild_index` call.

L4. **Single node:** no replication or failover for the server or FAISS indices.

L5. **MAC Flood detection:** more effective with a dedicated time-windowed MAC entropy counter for the current implementation receives frames but does not track an insertion clock.

L6. **VLAN Hopping — not empirically validated:** the `VLANTracker` module is fully implemented and the scoring logic is structurally sound (a double-tagged frame is a binary observable), but detection thresholds have not been calibrated against real capture data. Validation requires a managed switch with a Dot1Q trunk port, which was unavailable in the test environment. False-positive behaviour on legitimate QinQ (`0x88A8`) traffic is also uncharacterised.

10 Conclusions

Sairene achieves reliable detection of 11 empirically validated attack families and implements detection logic for a 12th (VLAN Hopping) in a controlled LAN environment without any external dependencies. Three complementary layers operate hierarchically:

1. **L2 multi-signal heuristic scores** (ARPTracker, DHCPTracker, MACFloodTracker, STPTracker, VLANTracker)
2. **Behavioural L3/L4 flows** with forward-only IAT and intelligent 3-tuple aggregators
3. **IsolationForest** as the statistical backbone

The *promotion discipline* - the system's signature architectural choice - prevents false-alert storms while ensuring that every confirmed anomaly has a sufficient evidentiary basis (`flow_summary` or `l2_summary`). Competing hypotheses (e.g. ARP poisoning vs. GARP storm) enable precise diagnosis with distinct response directives.

The integration of a RAG-augmented LLM (Ollama Qwen 2.5) allows operators without deep packet-analysis expertise to perform forensic Q&A in natural language, with results grounded in deterministic anomaly scoring rather than LLM hallucination [Lewis et al., 2020].

10.1 Future Directions

- **VLAN Hopping empirical validation:** deploy `VLANTracker` against a physical managed-switch trunk-port environment to calibrate thresholds and characterise false-positive behaviour on legitimate QinQ traffic
- **Live packet capture** via PF_RING / AF_PACKET for real-time operation
- **BERT-based L7 classifier** for deeper application-layer analysis [Devlin et al., 2018]
- **Federated IsolationForest training** across organisations without sharing raw traffic [Li et al., 2020]
- **Graph-based correlation** to detect multi-stage attacks spanning multiple flows [Wang et al., 2022]
- **Adaptive thresholds** using online learning to reduce false positives over time [Chandola et al., 2009]

References

Anderson, R. (2008). *Security Engineering: A Guide to Building Dependable Distributed Systems* (2nd ed.). Wiley.

Axelsson, S. (2000). Intrusion detection systems: A survey and taxonomy. *Technical Report 99-15*, Chalmers University of Technology.

Biondi, P. (2010). *Scapy: Explore the net with new eyes.*

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*

Convery, S., & Miller, D. (2002). VLAN security white paper: Cisco Catalyst switch family. *Cisco Systems Technical White Paper.*

Dempster, A. P. (1968). A generalization of Bayesian inference. *Journal of the Royal Statistical Society, Series B.*

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding.

Eddy, W. M. (2007). TCP SYN flooding attacks and common mitigations. *RFC 4987*, IETF.

Garcia-Teodoro, P., Diaz-Verdejo, J., Macia-Fernandez, G., & Vasquez, E. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges.

Gu, G., Perdisci, R., Zhang, J., & Lee, W. (2008). BotMiner: Clustering analysis of network traffic for protocol- and structure-independent botnet detection. *Proceedings of USENIX Security Symposium.*

Han, S., & Kim, H. (2015). Detecting ICMP tunneling traffic using payload analysis. *Journal of Information Security and Applications.*

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data.*

Kruegel, C., & Vigna, G. (2003). Anomaly detection of web-based attacks. *Proceedings of ACM CCS.*

Lewis, P., Perez, E., Piktus, A., Petroni, F., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.

Li, T., Sahu, A. K., Talwalkar, A., & Smith, V. (2020). Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine.*

Lippmann, R., Haines, J. W., Fried, D. J., Korba, J., & Das, K. (2000). The 1999 DARPA off-line intrusion detection evaluation.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM).*

Overtone, D. (2012). *iodine: IP-over-DNS tunnel*.

Paxson, V. (1999). Bro: A system for detecting network intruders in real-time. *Computer Networks*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*.

Postel, J. (1981). Transmission Control Protocol. *RFC 793*, IETF.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP*.

Roesch, M. (1999). Snort: Lightweight intrusion detection for networks. *Proceedings of LISA '99*.

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *Proceedings of IEEE Symposium on Security and Privacy*.

Stalmans, E., & Irwin, B. (2011). A framework for DNS based detection and mitigation of malware infections on a network. *Proceedings of ISSA 2011*.

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of IEEE CISDA*.

Wang, Q., Hassan, W. U., Bates, A., & Gunter, C. (2022). THREATRACE: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security*.

Watkins, L., et al. (2020). Detecting Cobalt Strike beacons. *Journal of Cybersecurity*.

Whalen, S. (2001). An introduction to ARP spoofing. *Node99 White Paper*.

Yersinia Project. (2005). *Yersinia: A framework for layer 2 attacks*.

OWASP. (n.d.). *DHCP Starvation Attack*.

SANS Institute. (2003). *Understanding MAC Flooding and CAM Table Attacks*. SANS Reading Room.

Lyon, G. F. (2009). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Insecure.Com LLC.

Strayer, W. T., Walsh, R., Livadas, C., & Lapsley, D. (2008). Detecting botnets with tight command and control. *Proceedings of IEEE LCN*.